Ενότητα 06:
The Pipeline I

ΕΕΙ 233: ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΑΝΔΡΕΑΣ ΔΙΑΒΑΣΤΟΣ

andreas.diavastos@cut.ac.cy

Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

CARL

1

Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

Outline

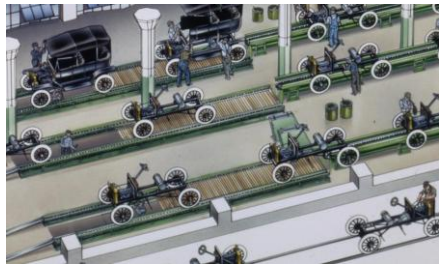
- Pipeline overview
- Επίδοση pipeline
- Pipeline Hazards
- Πως τα αντιμετωπίζουμε
- Instruction-level Parallelism Optimizations (Summary)

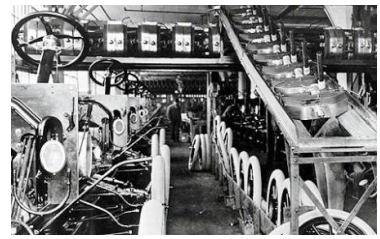
- Υλικό από Κεφάλαιο 6 του βιβλίου του μαθήματος

2

Ιστορικά

- Γραμμή παραγωγής δημιουργήθηκε από τον Henry Ford (1910s)
- Αύξησε το ρυθμό παραγωγής αυτοκινήτων ανά μονάδα χρόνου (Throughput)

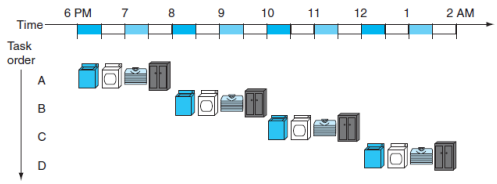


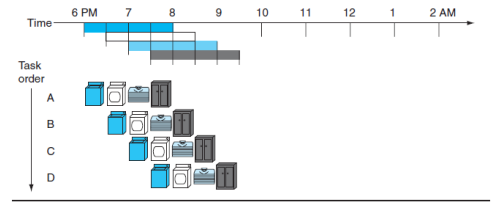




Pipeline (Διασωλήνωση)

- Είναι μια τεχνική που χρησιμοποιείται και μέσα στους επεξεργαστές για τη βελτίωση της επίδοσης, σπάζοντας το Datapath του επεξεργαστή σε μικρότερα στάδια (pipeline stages)

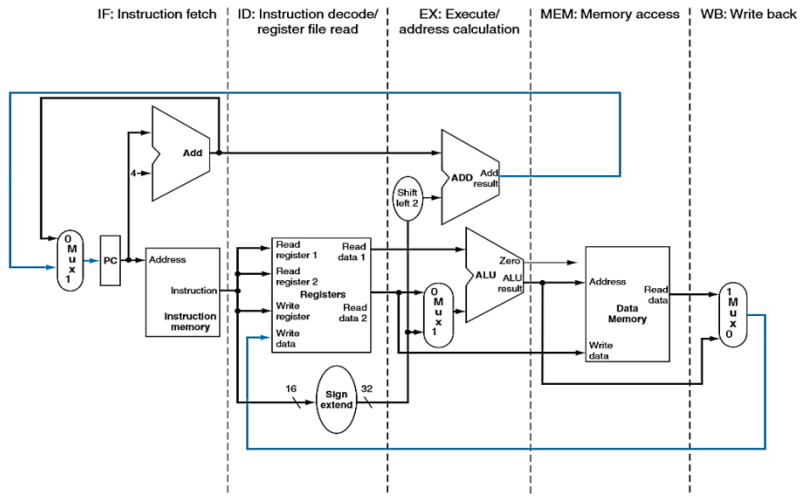




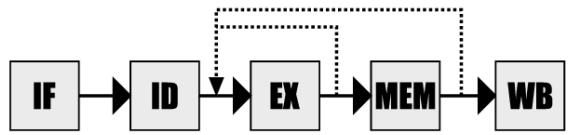
- Χωρίς pipeline → 8 ώρες
- Με pipeline → 3.5 ώρες

$$\text{Speedup} = \frac{8\ \text{ώρες}}{3.5\ \text{ώρες}} = 2.28\times$$

MIPS Pipeline Datapath



MIPS (RISC) Processor Basic 5-stages pipeline



IF: Instruction fetch
ID: Instruction decode and register read
EX: Execute, address generation
MEM: Memory access
WB: Write back to registers

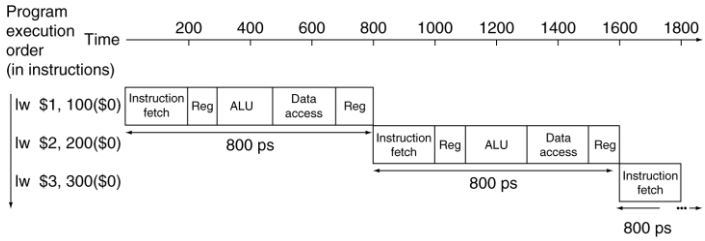
- Η εκτέλεση κάθε εντολής χωρίζεται σε στάδια
- Σε κάθε στάδιο ολοκληρώνεται ένα κομμάτι της εντολής
- Κάθε στάδιο λειτουργεί ανεξάρτητα και **παράλληλα** από τα υπόλοιπα στάδια
- Κάθε στάδιο μπορεί να παίρνει διαφορετικό "χρόνο" να εκτελεστεί
- Μπορώ να εκτελώ ταυτόχρονα πολλές διαφορετικές εντολές → Αύξηση του throughout και συνεπώς της επίδοσης της εφαρμογής (όχι της εντολής!)

Επίδοση του Pipeline

- Υποθέστε ότι ο χρόνος για:
  - Register File read/write → 100ps
  - Για όλα τα υπόλοιπα στάδια → 200ps

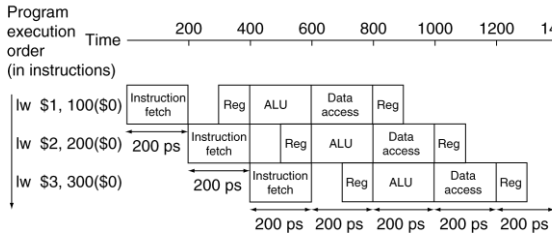
Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
lw	200ps	100 ps	200ps	200ps	100 ps	800ps
sw	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
beq	200ps	100 ps	200ps			500ps

Κάθε πόση ώρα τελειώνει μια εντολή;



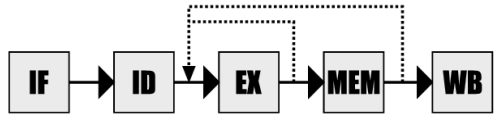
Single-cycle (T_c= 800ps)

Κάθε κύκλο τελειώνει μία εντολή!



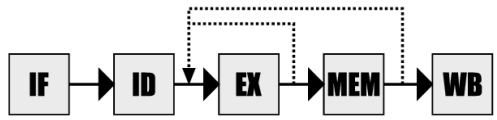
Pipelined (T_c= 200ps)

Επίδοση του Pipeline



- Ιδανικός Χρόνος μεταξύ Εντολών$_{με\ pipeline}$ $= \frac{\text{Χρόνος μεταξύ Εντολών}_{\text{χωρίς pipeline}}}{\text{Αριθμός Pipeline σταδίων}}$
- Ιδανικός → Τα στάδια είναι balanced (δλδ, όλα έχουν τον ίδιο **χρόνο** εκτέλεσης)
- $Ideal\ Speedup = \frac{\text{Χρόνος μεταξύ Εντολών}_{\text{χωρίς pipeline}}}{\text{Χρόνος μεταξύ Εντολών}_{\text{με pipeline}}} = \sim$Αριθμός Pipeline σταδίων
- Παράγοντες που επηρεάζουν τον συνολικό χρόνο εκτέλεσης σε ένα pipeline:
  - Ο χρόνος εκτέλεσης κάθε σταδίου (ιδανικά ο ίδιος σε κάθε στάδιο, αλλά δεν συμβαίνει πάντα)
  - Ο αριθμός των σταδίων του pipeline (Περισσότερα ή Λιγότερα?)
  - Καθυστερήσεις στο pipeline (Pipeline Hazards)
  - Χρόνος να γεμίσουμε το pipeline

Επίδοση του Pipeline (χωρίς stalls)



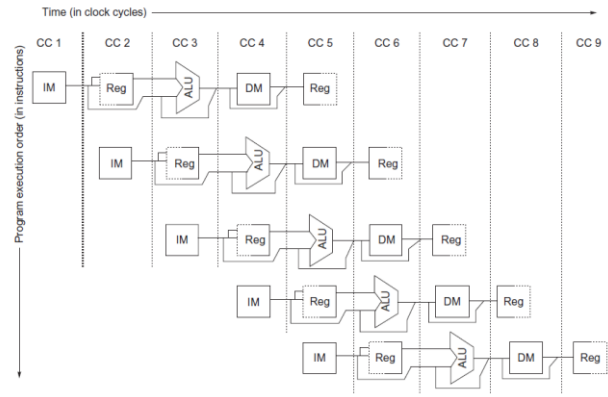
- $Clock\ Cycle\ pipelined\ = Time\ of\ slowest\ pipeline\ stage$

Από που προέρχεται η βελτίωση της επίδοσης??

- Από την αύξηση του throughput (ρυθμό εκτέλεσης εντολών ανά μονάδα χρόνου)
- Ο χρόνος εκτέλεσης ανά εντολή ΔΕΝ μειώνεται!!
  - Ακόμα χειρότερα, κάποτε ίσως και να αυξάνεται!

Pipeline (Datapath) in a Processor

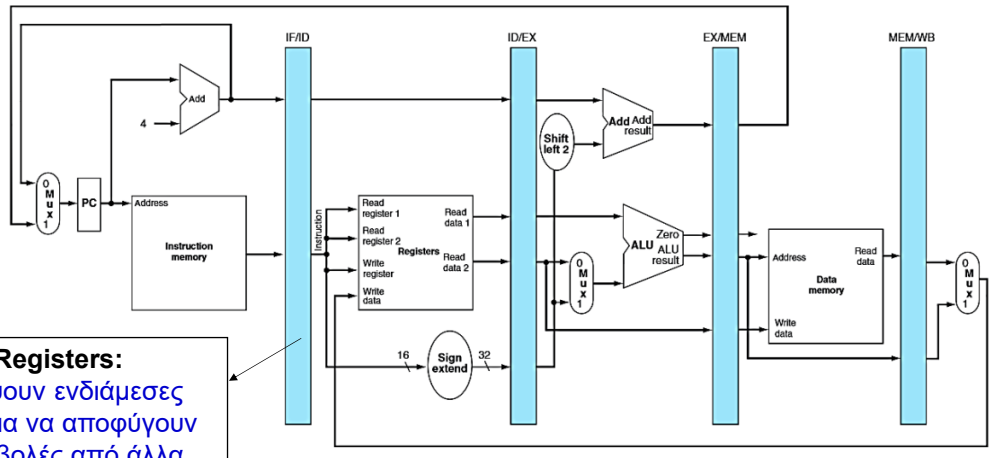
- Κάθε εντολή πρέπει να περάσει από όλα τα στάδια (ανεξαρτήτως τύπου)
- Χρειάζονται 5 κύκλοι για να βγει μια εντολή από το pipeline
- Δυνατότητα να ολοκληρώνεται μια εντολή ανά κύκλο (ιδανικά!)
- Ο average χρόνος μεταξύ δύο εντολών είναι ο χρόνος εκτέλεσης του σταδίου με το μεγαλύτερο χρόνο εκτέλεσης



Pipelining and ISA Design

- Η MIPS ISA σχεδιάστηκε για να υποστηρίζει pipeline
  - Όλες οι εντολές είναι 32 bits
    - Πιο γρήγορο να φέρνουμε και να αποκωδικοποιήσουμε τις εντολές
    - Π.χ. η x86 ISA έχει εντολές μεγέθους από 1- μέχρι 17-bytes
  - Πιο λίγες και απλές εντολές
    - Μπορεί να αποκωδικοποιήσει εντολές και να διαβάσει τους Registers σε ένα βήμα

Pipeline Datapath – Pipeline Registers



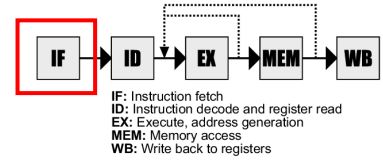
Pipeline Registers:
Αποθηκεύουν ενδιάμεσες τιμές για να αποφύγουν παρεμβολές από άλλα στάδια

Στάδιο 1: Instruction Fetch (IF)

- Χρησιμοποιούμε το Program Counter (PC) για να φέρουμε (fetch) την εντολή που πρέπει να εκτελεστεί
- Αυξάνουμε το PC για να δείχνει στην επόμενη εντολή

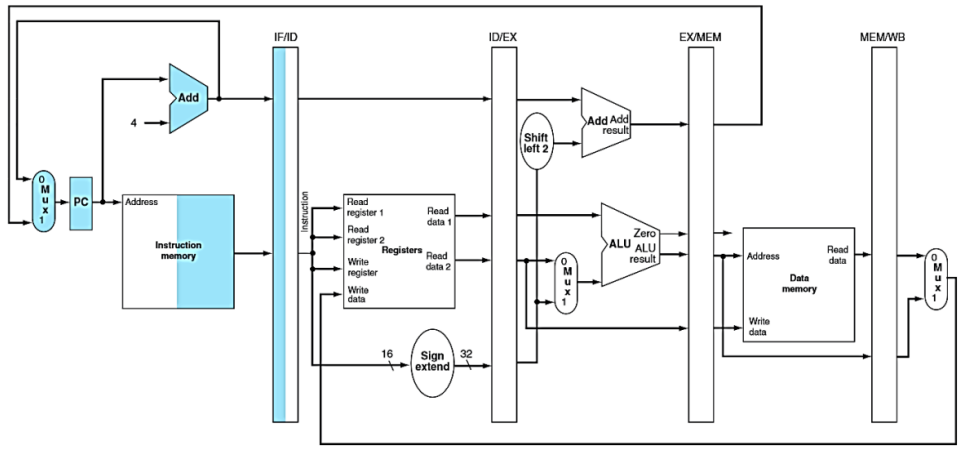
```
lw $rt, immed($rs)
```

31-26	25-21	20-16	15-0
opcode	rs	rt	immed



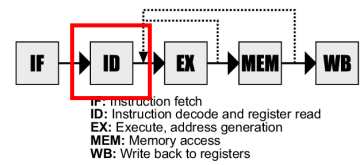
IF: Instruction fetch
ID: Instruction decode and register read
EX: Execute, address generation
MEM: Memory access
WB: Write back to registers

Στάδιο 1: Instruction Fetch (IF)

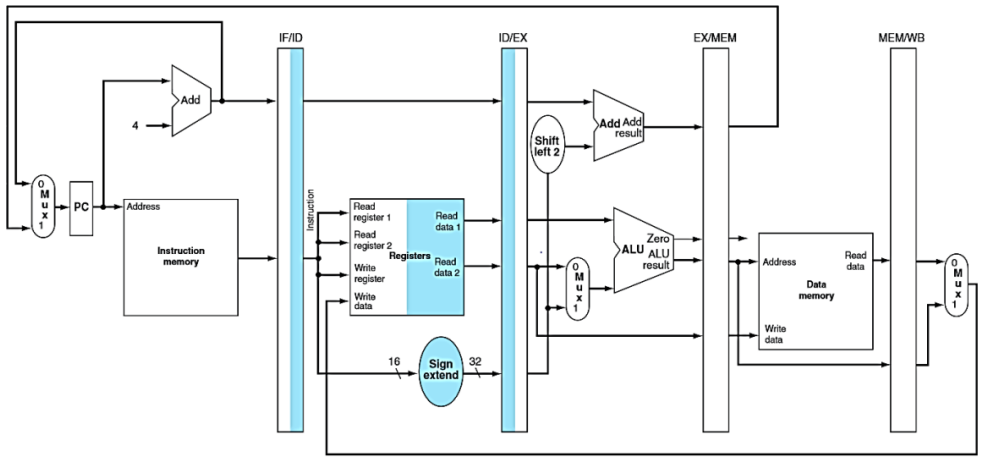


Στάδιο 2: Instruction Decode (ID)

- Αποκωδικοποίηση της εντολής
- Προετοιμάζει την εντολή για εκτέλεση
- Διαβάζουμε τους source registers (δεδομένα)
- Εάν η εντολή είναι Branch τότε μετέτρεψε σε 32 bits το offset της διεύθυνσης που θα δείχνει το Branch και θα πάει η εκτέλεση εάν το Branch είναι TRUE (sign extend)

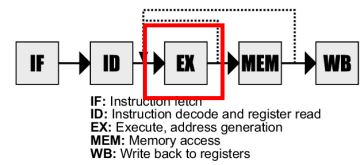


Στάδιο 2: Instruction Decode (ID)

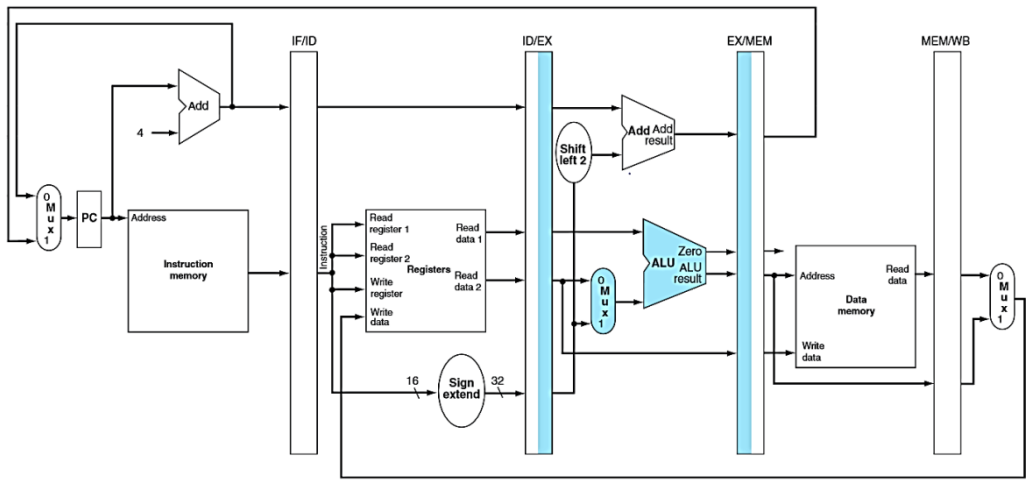


Στάδιο 3: Execute (EX)

- Οι μονάδες εκτέλεσης του επεξεργαστή (π.χ. ALU) εκτελούν τις απαραίτητες πράξεις αναλόγως της εντολής:
  - Memory Reference
    - Υπολογίζει την διεύθυνση από την οποία θα φέρει/γράψει δεδομένα στο επόμενο στάδιο όταν η εντολή είναι `LOAD` ή `STORE`
  - Register-Register ALU instruction (R-type)
    - Εκτελεί την εντολή βάση του opcode πάνω στους δύο source registers
  - Register-Immediate ALU instruction
    - Εκτελεί την εντολή βάση του opcode πάνω στον ένα register και στην μια σταθερά
  - Conditional Branch
    - Εκτελεί τον έλεγχο του Branch για να βρει εάν είναι TRUE ή FALSE
    - Υπολογίζει την διεύθυνση που θα πάει εάν είναι TRUE

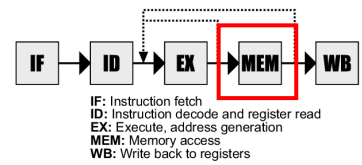


Στάδιο 3: Execute (EX)



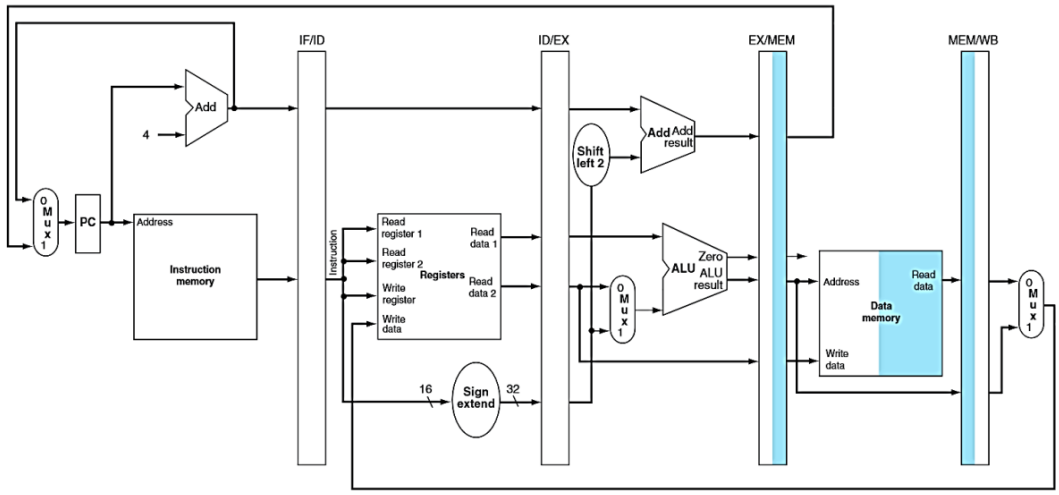
Στάδιο 4: Memory Access (MEM)

- Εάν η εντολή είναι `LOAD`
  - Διαβάζουμε τα δεδομένα από την μνήμη χρησιμοποιώντας την διεύθυνση που υπολογίσαμε στο προηγούμενο στάδιο
- Εάν η εντολή είναι `STORE`
  - Γράφουμε τα δεδομένα προς την μνήμη χρησιμοποιώντας την διεύθυνση που υπολογίσαμε στο προηγούμενο στάδιο



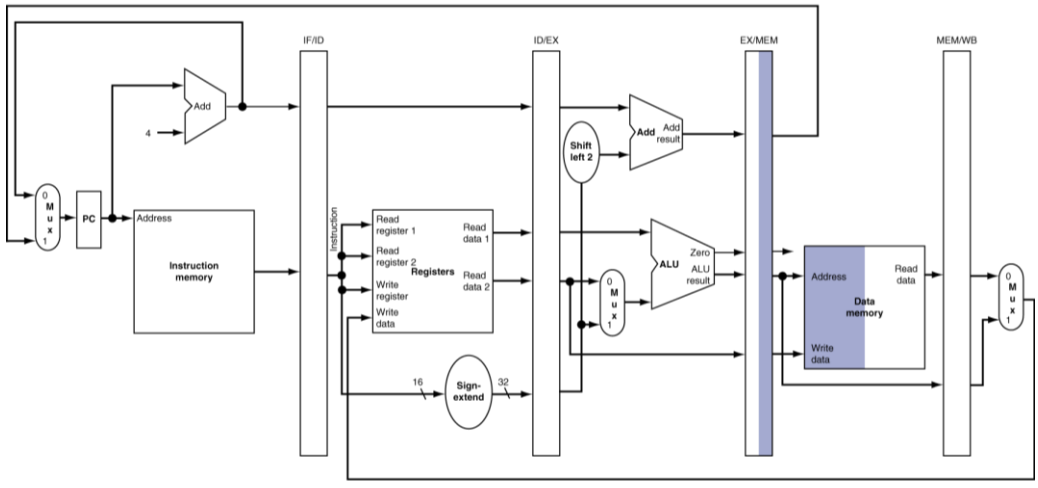
IF: Instruction fetch
ID: Instruction decode and register read
EX: Execute, address generation
MEM: Memory access
WB: Write back to registers

Στάδιο 4: Memory Access (MEM for LOAD)



21

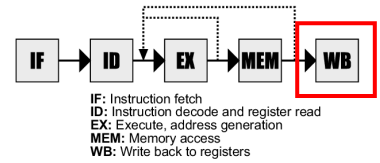
Στάδιο 4: Memory Access (MEM for STORE)



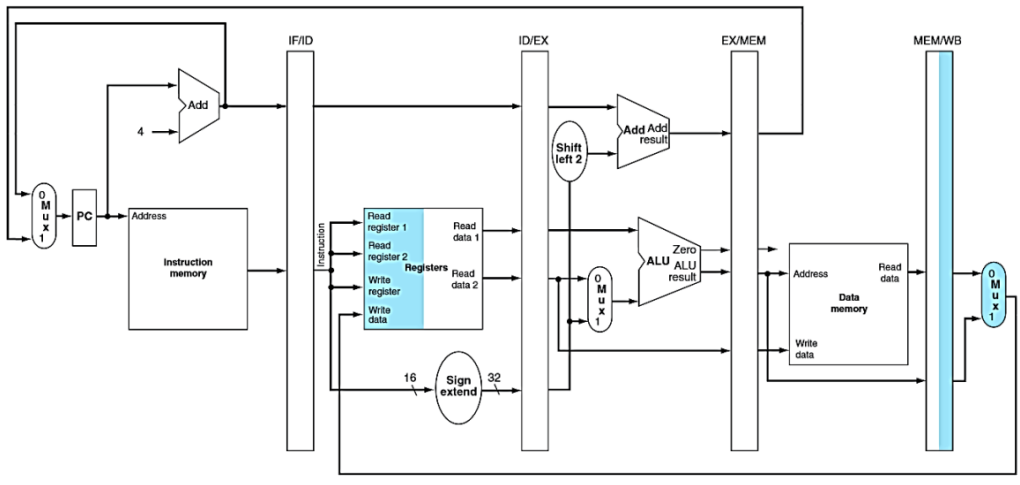
22

Στάδιο 5: Write-back (WB)

- Register-Register ALU instruction
- `LOAD` instruction
- Γράφει το αποτέλεσμα της εντολής στο Register File



Στάδιο 5: Write-back (WB)



Ενότητα 06:
The Pipeline I

ΕΕΙ 233: ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΑΝΔΡΕΑΣ ΔΙΑΒΑΣΤΟΣ

andreas.diavastos@cut.ac.cy

Τεχνολογικό Πανεπιστήμιο Κύπρου
Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

CARL

25